SDR

Introduction à Go

Olivier Lemer

Sommaire

1. Comparaison avec Java

2. Ressources et Développement

3. Bases du langage

4. Héritage et Composition

5. Concurrence

6. Programmation TCP-IP

↓ Comparaison avec Java

Comparaison avec Java

Histoire

2007 - Créé en interne pour Google, par Robert Griesemer, Rob Pike et Ken Thompson.

2009 - Devient Open Source avec communauté très active.

2012 - Publication de la version 1.0, stable.

Pensé pour

- **Logiciels serveurs**
- **Scalability** (10^6 machines)
  (Concurrence builtin au langage)

Comparaison avec Java

Similarités

- Typage statique fort
- Garbage Collector
- Memory safety partielle (nulls, out-of-bounds)
- Variables toujours initialisées avec leur "zéro"
- Méthodes
- Interfaces
- Assertion sur les types (`instanceof`)
- Introspection (reflection)
- Closures et captures

Différences

- Pas de VM *(compilation vers un executable)*
- Librairies incluses dans executable (static linking)
- Fonctions as first class citizens
- Strings, Maps, Arrays, Slices sont builtin
- Concurrence est builtin
- **Pas de classes**
  - Pas de constructeurs
  - Pas d'héritage *(mais : composition)*
- Pas d'implémentation par défaut des méthodes d'une interface
- Pas d'exceptions *(sauf pour abort)*
- Pas d'annotations *(mais tags dans structs)*

Philosophie de Go : lisibilité et de simplicité, afin d'éviter les erreurs d'implémentations.

Comparaison avec Java

Hello, World!

main.java

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

hello.go

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```



Point virgule optionnel

Ressources et développement

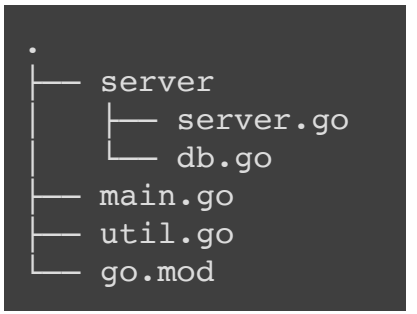
Ressources et Développement

Sur le site officiel : *go.dev*

- Édition et exécution de code en ligne
- Tutoriels et exercices de prise en main *(Learn > Tour of Go)*
- Installation compilateur, librairies et utilitaires *(Docs > Installing Go)*
- Organisation d'un projet *(Docs > How to Write Go Code)*
- Outils de développement *(Docs > Editor plugins and IDEs)*
- Bonnes pratiques *(Docs > Effective Go)*
- Et autres références, articles, vidéos ...

Ressources et Développement

Structure d'un projet



Go module

- Contient le fichier `go.mod`
- Regroupe plusieurs packages en un projet.
- Requis depuis le passage aux modules, même pour un seul package.

Main package

Le package à la racine du module est spécial et doit contenir une fonction `main()`.

Go packages

- Chaque dossier constitue un package du même nom.
- Tous fichiers d'un même package partagent tout sans besoin d'import explicite.
- Chaque fichier commence par `package <name>` pour indiquer son package.
- Le nom des fichiers est sans importance.

Export entre packages

Variable/Fonction avec **`Majuscule`** : identifiant exporté.

Variable/Fonction avec **`minuscule`** : identifiant *pas* exporté.

Ressources et Développement

Structure d'un projet

`/hello.go`

```go
package main

import "fmt"

func main() {
    fmt.Println(reverse("Hello, World!"))
}
```

`/util.go`

```go
package main

func reverse(s string) string {
    b := []rune(s)
    for i := 0; i < len(b)/2; i++ {
        j := len(b) - i - 1
        b[i], b[j] = b[j], b[i]
    }
    return string(b)
}
```

Notez : pas besoin d'importer `util.go` dans `hello.go`, car ils sont dans le même package.

Ressources et Développement

Structure d'un projet

Même exemple avec plusieurs packages.

`/hello.go`

```go
package main

import (
    "fmt"
    "module_name/util"
)

func main() {
    fmt.Println(util.Reverse("Hello, World!"))
}
```

Package `util` importé

`/util/util.go`

```go
package util

func Reverse(s string) string {
    b := []rune(s)
    for i := 0; i < len(b)/2; i++ {
        j := len(b) - i - 1
        b[i], b[j] = b[j], b[i]
    }
    return string(b)
}
```

`Reverse` avec majuscule => exporté

Ressources et Développement

Structure conventionnelle d'un projet

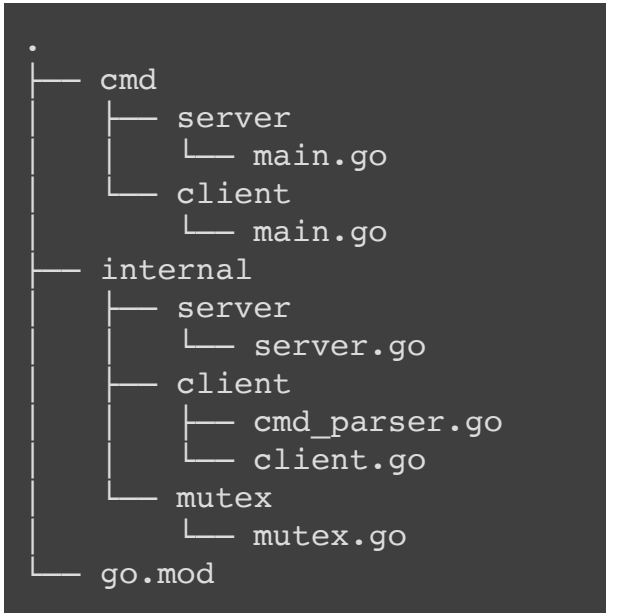
Packages cmd

Contient uniquement les sources d'exécutables, qui référencent la logique interne.

Généralement très légers.

Packages internes

Modularisent la logique interne.



Ressources et Développement

Import de packages externes

`go get [url]`

Obtient le package pointé par l'url, et crée une dépendance à celui-ci dans `go.mod`.

```
go get github.com/example
```

Le package peut ensuite être importé et utilisé dans tout fichier du module.

```
import "github.com/example"
```

Nous ne le ferons pas dans les labos de ce cours.

Ressources et Développement

Compiler et installer

`go run [package]`

Compile et exécute un package (ou un fichier formant un package) d'éxécutable.

```
go run hello.go
```

`go build [package]`

Compile un package (ou un fichier formant un package) en un executable dans le dossier courant.

```
go build hello.go
```

`go install [package]`

Compile un package (ou un fichier formant un package), puis installe l'executable dans `$GOBIN`, donc `$HOME/go/bin/` par défaut.

```
go install
```

Ressources et Développement

Tester

`go test`

Compile et execute **toute fonction `Test*`** dans **tout fichier `*_test.go`**.

```
go test
```

`util/util_test.go`

```
package util

import "testing"

func TestSomething(t *testing.T) {
    // ...
}
```



Bases du langage

Collections

Bases du langage

Types de base

```
bool

string

int  int8  int16  int32  int64
uint uint8 uint16 uint32 uint64 uintptr

byte // alias for uint8

rune // alias for int32
     // represents a Unicode code point

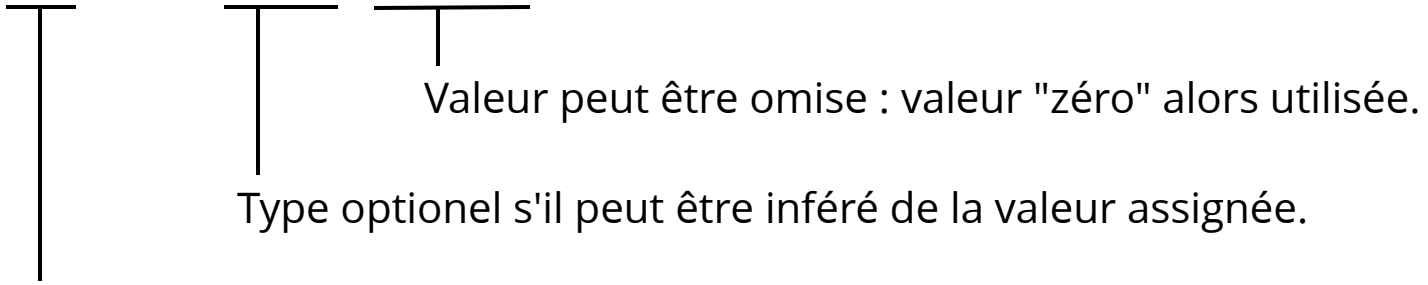
float32 float64

complex64 complex128
```

Bases du langage

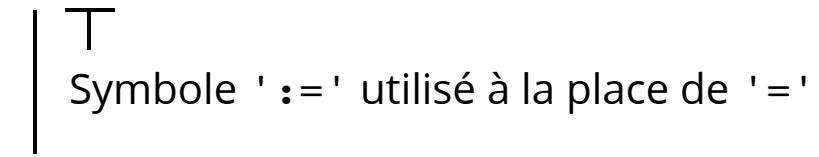
Variables

```
var <name> <type> = <value>
```



Forme équivalente succincte *(uniquement pour variables locales muables)*

```
<name> := <value>
```



Bases du langage

Tableaux et slices

Syntaxe du type d'un tableau

```
[n]type // Type d'un array de taille n et de type type
```

Syntaxe d'un littéral tableau

```
[2]string{"Hello", "there"}
```

Syntaxe d'un slice

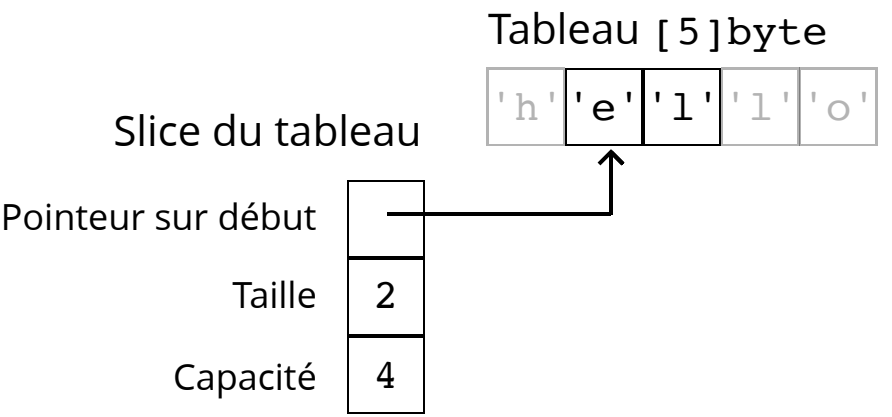
```
tableau[a:b] // slice entre a (incl) et b (excl) de tableau
```

```
tableau := [5]byte{'h','e','l','l','o'}

slice := tableau[1:3] // ['e', 'l']

slice = append(slice, 'i') // ['e', 'l', 'i']
```

Augmente la taille et insère la valeur.
Si le tableau est trop petit, en crée un nouveau et pointe sur celui-là.



Bases du langage

Tableaux et slices

Syntaxe du type d'un tableau

```
[n]type // Type d'un array de taille n et de type type
```

Syntaxe d'un littéral tableau

```
[2]string{"Hello", "there"}
```

Syntaxe d'un slice

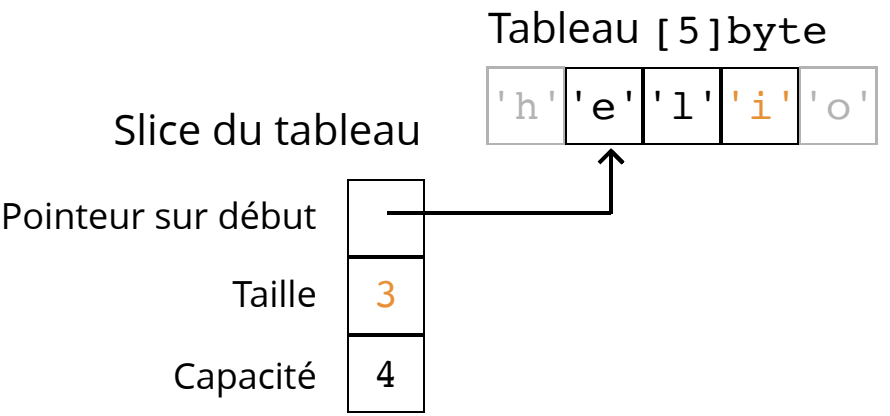
```
tableau[a:b] // slice entre a (incl) et b (excl) de tableau
```

```
tableau := [5]byte{'h','e','l','l','o'}

slice := tableau[1:3] // ['e', 'l']

slice = append(slice, 'i') // ['e', 'l', 'i']
```

Augmente la taille et insère la valeur.
Si le tableau est trop petit, en crée un nouveau et pointe sur celui-là.



***Note** : modifier la slice modifie le tableau sous-jacent.*

Bases du langage

Tableaux et slices - Question

```
 1 tableau := [5]byte{'h','e','l','l','o'}
 2
 3 slice := tableau[0:3] // ['h', 'e', 'l']
 4
 5 slice[0] = 'R'
 6
 7 slice = append(slice, 'i')
 8 slice = append(slice, 'u')
 9 slice = append(slice, 'm')
10
11 slice[0] = 'H'
```

Que contient `tableau` ?

1. Heliu
2. Helium
3. heliu
4. helium
5. Reliu
6. Relium

Bases du langage

Tableaux et slices - Question

```
 1 tableau := [5]byte{'h','e','l','l','o'}
 2
 3 slice := tableau[0:3] // ['h', 'e', 'l']
 4
 5 slice[0] = 'R'
 6
 7 slice = append(slice, 'i')
 8 slice = append(slice, 'u')
 9 slice = append(slice, 'm')
10
11 slice[0] = 'H'
```

Que contient `tableau` ?

1. Heliu
2. Helium
3. heliu
4. helium
5. Reliu
6. Relium

Initialement, la slice pointe sur le tableau. Tout changement sur la slice s'applique donc au tableau, y inclus les append.

Lorsqu'on tente d'append au delà de la capacité du tableau ('m'), un nouveau tableau est créé et la slice pointe dessus.

À partir de ce moment, tout changement se fait sur ce nouveau tableau, `'R'` n'est donc pas remplacé par `'H'` ligne 11.

Bases du langage

Tableaux et slices

Syntaxe du type d'un tableau

```
[n]type // Type d'un array de taille n et de type type
```

Syntaxe d'un littéral tableau

```
[2]string{"Hello", "there"}
```

Syntaxe d'un slice

```
tableau[a:b] // slice entre a (incl) et b (excl) de tableau
```

Création d'un slice avec `make`

```
make([]type, len, cap) // slice de type type, longueur len et capacité cap
```

Bases du langage

Tableaux et slices

Syntaxe du type d'un tableau

```
[n]type // Type d'un array de taille n et de type type
```

Syntaxe d'un littéral tableau

```
[2]string{"Hello", "there"}
```

Syntaxe d'un slice

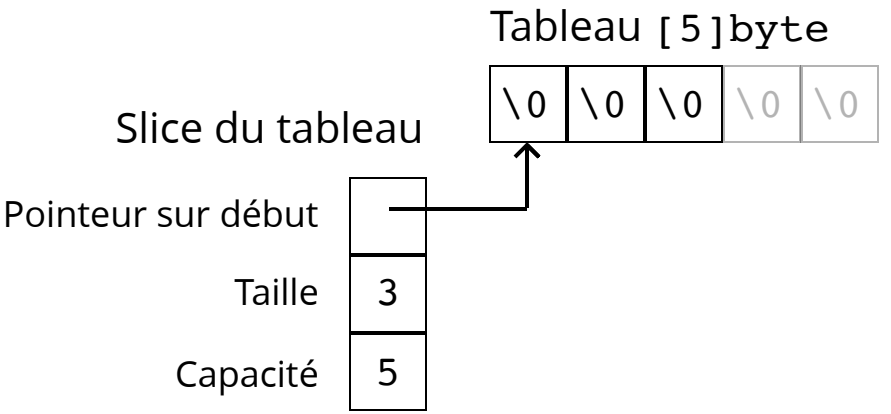
```
tableau[a:b] // slice entre a (incl) et b (excl) de tableau
```

Création d'un slice avec `make`

```
make([]type, len, cap) // slice de type type, longueur len et capacité cap
```

```
slice := make([]byte, 3, 5)
```

Crée un tableau de zéros de taille 5,
et crée un slice de longueur 3 dessus.



Bases du langage

Maps

Syntaxe du type d'une map

```
map[int]string // Type d'une map de clés int et valeurs string
```

Syntaxe d'un littéral map

```
map[int]string{1: "un", 2: "deux"}
```

Création d'une map avec `make`

```
make(map[int]string) // map vide de int vers string
```

Modification d'une map

```
nombres := make(map[int]string)

nombres[1] = "un"  // Ajout d'une valeur
nombres[1] = "one" // Remplacement d'une valeur

un := nombres[1]   // Lecture d'une valeur

delete(nombres, 1) // Suppression d'une valeur

un, ok := nombres[1] // Lecture et vérification d'une valeur
```

⊤

`true` ssi la valeur existe dans la map.

Bases du langage

Sets

Bases du langage

Sets

N'existe pas.

Bases du langage

Sets

N'existe pas.

Map largement suffisant : la valeur peut être une structure vide.

```
noms := make(map[string]bool)
mots := make(map[string]struct{})
```



Bases du langage

Conditions et boucles

Bases du langage

Instructions conditionnelles

Statement court *(optionnel)*
Comme pour `for`, permet d'initialiser une variable qui existera dans le corps du `if` et du `else`.

```
1 if <init>; <cond> {
2     // ...
3 } else if {
4     // ...
5 } else {
6     // ...
7 }
```

Pas de parenthèses nécessaires pour la condition.

Blocs `else` et `else if` optionnels

Bases du langage

Instructions conditionnelles

Statement court *(optionnel)*
Comme pour le `if`.

```
switch <init>; <value> {
case <value> :
    // ...
    fallthrough
case <value> :
    // ...
default :
    // ...
}
```

`break` par défaut.
Pour forcer l'évaluation du `case` suivant, utiliser `fallthrough`

Bloc `default` optionnel.

Switch sur une valeur

```
switch {
case <cond> :
    // ...
case <cond> :
    // ...
default :
    // ...
}
```

Si la valeur est omise, alors les `case` prennent des conditions.

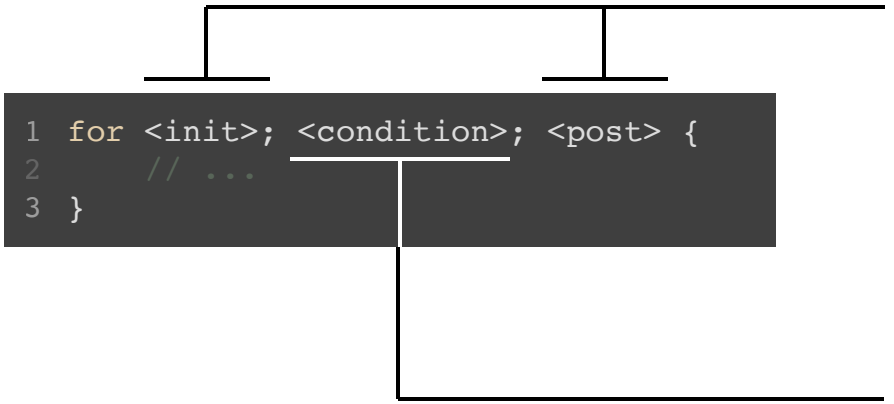
On obtient alors un équivalent exact de `if-elseif-elseif-else`

Switch sur rien

Bases du langage

Boucles

Boucle `for`



```
for <init>; <condition>; <post> {
    // ...
}
```

`<init>` et `<post>` sont optionnels

```
for ; <condition>; {
```

Lorsque les deux sont omis, les `';'` peuvent l'être aussi.

```
for <condition> {
```

`<condition>` est optionnel.

```
for { // infinite loop
```

Boucle `foreach`



```
for i, v := range <iterable> {
    // ...
}
```

Mot-clé `range` fait partie de la syntaxe.

Peut être un array ou une map

Donne l'index (ou la clé) et la valeur. Utiliser le symbole `'_'` pour ne pas récupérer dans une variable.

Bases du langage

Boucles

Boucle `while`

Bases du langage

Boucles

Boucle `while`

N'existe pas.

Bases du langage

Boucles

Boucle `while`

N'existe pas.

Boucle `for` complètement équivalente

```
for <condition> {
```



Bases du langage

Fonctions

Bases du langage

Fonctions

```
func compare[T comparable](a T, b T) bool {
```

Paramètres de type

(Fonctions génériques)

- Optionnel
- Liste de paramètres de forme

```
<T> <constraints>
```

  - `<T>` le nom du paramètre
  - `<constraints>` décrit des contraintes sur le type

Paramètres

- Liste de paramètres de forme

```
<name> <type>
```

- Le type peut être omis s'il est celui du paramètre suivant :

```
func add(x, y int) int {
```

Type de retour

- Optionnel si la fonction ne retourne rien (pas de `void`)
- Pour retourner plusieurs valeurs

```
func origin() (int, int) {
```

- Valeurs retournées peuvent être nommées

```
func origin() (x int, y int) {
    x = 1;
    y = 2;
    return
}
```

Elles seront alors des variables dans la fonction, et seront retournées automatiquement.

  - Si deux partagent leur type, le premier peut l'omettre

```
func origin() (x, y int) {
```

Bases du langage

Fonctions - exemples

```
1 func add(x, y int) int { // type de x omis car identique à y
2     return x + y
3 }
```

```
1 func swap(x, y string) (string, string) { // Plusieurs valeurs retournées
2     return y, x
3 }
```

```
a, b := swap("Hello", "world!"); // Attribution à deux variables
_, _ = swap("Hello", "world!"); // Possibilité d'ignorer. Nombre doit correspondre.
```

```
1 func split(sum int) (x, y int) { // Valeurs retournées nommées.
2     x = sum * 4 / 9              // Variables x et y déjà existantes.
3     y = sum - x
4     return                       // Retourne x et y automatiquement.
5 }
```

Bases du langage

Fonctions comme *first class citizens*

Syntaxe du type d'une fonction

```
func(<parameter_types>) <return_types>
```

Syntaxe d'une fonction anonyme

```
func(<parameters>) <return_types> { <body> }
```

Exemple

```
 1 func square(n int) int {return n * n}
 2
 3 func main() {
 4     var f func(int) int                      // Variable de type func(int) int
 5
 6     f = func(n int) int { return -n }        // Attribution avec fonction anonyme
 7
 8     f = square                               // Attribution avec fonction existante
 9 }
10
11 func applyTwice(f func(int) int, n int) int { // Fonction comme argument
12     return f(f(n))
13 }
```

Bases du langage

Fonctions deviennent *closures*

Définition : objet représentant une fonction et l'environnement dans lequel elle a été créée.

```
 1 func getUidGenerator() func() int {
 2     var nextUid int = 0
 3     return func() int {
 4         nextUid += 1
 5         return nextUid
 6     }
 7
 8 }
 9
10 func main() {
11     genUid := getUidGenerator()
12     id1 := genUid() // 1
13     id2 := genUid() // 2
14 }
```

`nextUid` utilisée dans la fonction, elle est **capturée** par la closure.

`genUid`

Environment

```
nextUid: 0
```

function

```
nextUid += 1
return nextUid;
```

Bases du langage

Fonctions deviennent *closures*

Définition : objet représentant une fonction et l'environnement dans lequel elle a été créée.

```
 1 func getUidGenerator() func() int {
 2     var nextUid int = 0
 3     return func() int {
 4         nextUid += 1
 5         return nextUid
 6     }
 7
 8 }
 9
10 func main() {
11     genUid := getUidGenerator()
12     id1 := genUid() // 1
13     id2 := genUid() // 2
14 }
```

`nextUid` utilisée dans la fonction, elle est **capturée** par la closure.

`genUid`

Environment

```
nextUid: 1
```

function

```
nextUid += 1
return nextUid;
```

Bases du langage

Fonctions deviennent *closures*

Définition : objet représentant une fonction et l'environnement dans lequel elle a été créée.

```
 1 func getUidGenerator() func() int {
 2     var nextUid int = 0
 3     f := func() int {
 4         nextUid += 1
 5         return nextUid
 6     }
 7     nextUid = 42
 8     return f
 9 }
10
11 func main() {
12     genUid := getUidGenerator()
13     id1 := genUid() // 43
14     id2 := genUid() // 44
15 }
```

Si une variable capturée change,
elle change dans la closure aussi.

`genUid`

Environment

```
nextUid: 42
```

function

```
nextUid += 1
return nextUid;
```



Bases du langage

Structures et composition

Bases du langage

Déclaration de types

Déclarer un **nouveau type**

```
type <new_type> <existing_type>
```

Exemples

```
type intGenerator func() int

type image [][]uint32

type celsius float64
type fahrenheit float64

var ebulitionC celsius = 100
var ebulitionF fahrenheit = ebulitionC // ERROR : Cannot assign celsius to fahrenheit
```

Bases du langage

Déclaration de types

Déclarer un **nouveau type**

```
type <new_type> <existing_type>
```

Exemples

```
type intGenerator func() int

type image [][]uint32

type celsius float64
type fahrenheit float64

var ebulitionC celsius = 100
var ebulitionF fahrenheit = ebulitionC // ERROR : Cannot assign celsius to fahrenheit
```

Déclarer un **alias** pour un type donné

```
type <type_alias> = <existing_type>
```

Exemples

```
type celsius = float64
type fahrenheit = float64

var ebulitionC celsius = 100
var ebulitionF fahrenheit = ebulitionC // Correct ; même type : float64
```

Bases du langage

Structures

Définition : une structure est simplement un ensemble de champs nommés.

Syntaxe du type d'une structure

```
struct {
    <name1> <type1>
    <name2> <type2>
    // ...
}
```

Généralement associé à un nouveau type immédiatement

```
type Person struct {
    name string
    age  int
}

var fred Person
```

Mais reste un type comme un autre, peut être utilisé comme tel

```
var fred struct {
    name string
    age  int
};
```

Bases du langage

Structures

Définition : une structure est simplement un ensemble de champs nommés.

Syntaxe du type d'une structure

```
struct {
    <name1> <type1>
    <name2> <type2>
    // ...
}
```

Syntaxe de création d'une valeur de type struct

```
// Création d'une struct avec champs "zéros"
var fred Person // {"", 0}

// Création avec valeurs initiales fournies
var fred = Person{"fred", 99} // {"fred", 99}

// Création avec valeurs initiales fournies et nommées, ordre arbitraire
var fred = Person{age: 99, name: "fred"} // {"fred", 99}

// Création avec valeurs initiales partiellement fournies, "zéros" ailleurs
var fred = Person{name: "fred"} // {"fred", 0}
```

Bases du langage

Structures

Définition : une structure est simplement un ensemble de champs nommés.

Syntaxe du type d'une structure

```
struct {
    <name1> <type1>
    <name2> <type2>
    // ...
}
```

Syntaxe de création d'une valeur de type struct

```
// Création d'une struct avec champs "zéros"
var fred Person // {"", 0}

// Création avec valeurs initiales fournies
var fred = Person{"fred", 99} // {"fred", 99}

// Création avec valeurs initiales fournies et nommées, ordre arbitraire
var fred = Person{age: 99, name: "fred"} // {"fred", 99}

// Création avec valeurs initiales partiellement fournies, "zéros" ailleurs
var fred = Person{name: "fred"} // {"fred", 0}
```

Note: les règles d'export s'appliquent aux champs : pour qu'un champ soit public en dehors du package, il doit commencer par une majuscule.

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

Exemple

```
func (p Person) Majeur() bool {
    return p.age >= 18
}

if fred.Majeur() { // ...
```

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

Exemple

```
func (p Person) Majeur() bool {
    return p.age >= 18
}

if fred.Majeur() { // ...
```

Note : le récepteur n'est qu'un paramètre comme un autre.
Peut donc être *pensé* comme du **syntactic sugar** pour

```
func Majeur(p Person) bool {
    return p.age >= 18
}

if Majeur(fred) { // ...
```

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

`<type>` ne peut pas être un type builtin

```
func (s string) MyLength() int { // ERROR : string ne peut pas être récepteur
```

Bases du langage

Méthodes

Syntaxe d'une méthode : déclaration du récepteur de la fonction avant son nom

```
func (<name> <type>) <name> (<parametres>) <return_types> { // ...
```

`<type>` ne peut pas être un type builtin

```
func (s string) MyLength() int { // ERROR : string ne peut pas être récepteur
```

En revanche, `<type>` peut être un type équivalent à un type builtin

```
type MyString string

func (s MyString) MyLength() int { // Valide (!)
```

Bases du langage

Interfaces

Définition : représente la catégorie des types qui implémentent des méthodes données.

Syntaxe du type d'une interface

```
interface {
    <method1>
    <method2>
    // ...
}
```

```
type Walker interface {
    StepsPerSecond() int
}

type Person struct {
    // ...
}

func (p Person) StepsPerSecond() int { // Person est désormais un Walker
    // ...
}
```

Important

Les interfaces sont satisfaites **implicitement**, et non explicitement comme en Java.
Si une struct implémente les méthodes d'une interface, alors elle satisfait cette interface.

Bases du langage

Interfaces

Polymorphisme

```
var v <interface> = <valeur> // si <valeur> satisfait <interface>
```

Exemple

```
 1 type Walker interface {
 2     StepsPerSecond() int
 3 }
 4
 5 type Person struct { // ...
 6
 7 func (p Person) StepsPerSecond() int { // Person est désormais un Walker
 8     // ...
 9 }
10
11 func main() {
12     var p Person
13
14     var w Walker = p         // Polymorphisme
15 }
```

Bases du langage

Interfaces

Polymorphisme

```
var v <interface> = <valeur> // si <valeur> satisfait <interface>
```

Exemple

```
 1 type Walker interface {
 2     StepsPerSecond() int
 3 }
 4
 5 type Person struct { // ...
 6
 7 func (p Person) StepsPerSecond() int { // Person est désormais un Walker
 8     // ...
 9 }
10
11 func main() {
12     var p Person
13
14     var w Walker = p         // Polymorphisme
15 }
```

Bases du langage

Composition vs. héritage

Bases du langage

Composition vs. héritage : héritage

Bases du langage

Composition vs. héritage : héritage

```java
class Runner {
    // ...

    public void run(Task task) {
        task.run();
    }

    public void runAll(Task[] tasks) {
        for (Task task : tasks) {
            run(task);
        }
    }
}
```

```java
class CountingRunner extends Runner {
    private int count;

    // ...

    @Override public void run(Task task) {
        count++;
        super.run(task);
    }

    @Override public void runAll(Task[] tasks) {
        count += tasks.length;
        super.runAll(tasks);
    }
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

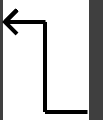
Bases du langage

Composition vs. héritage : héritage

```java
class Runner {
    // ...

    public void run(Task task) {
        task.run();
    }

    public void runAll(Task[] tasks) {
        for (Task task : tasks) {
            run(task);
        }
    }
}
```



```java
class CountingRunner extends Runner {
    private int count;

    // ...

    @Override public void run(Task task) {
        count++;
        super.run(task);
    }

    @Override public void runAll(Task[] tasks) {
        count += tasks.length;
        super.runAll(tasks);
    }
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

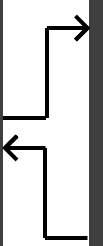
Bases du langage

Composition vs. héritage : héritage

```java
class Runner {
    // ...

    public void run(Task task) {
        task.run();
    }

    public void runAll(Task[] tasks) {
        for (Task task : tasks) {
            run(task);
        }
    }
}
```



```java
class CountingRunner extends Runner {
    private int count;

    // ...

    @Override public void run(Task task) {
        count++;
        super.run(task);
    }

    @Override public void runAll(Task[] tasks) {
        count += tasks.length;
        super.runAll(tasks);
    }
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

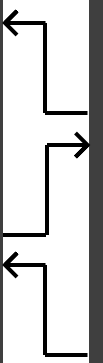
Bases du langage

Composition vs. héritage : héritage

```java
class Runner {
    // ...

    public void run(Task task) {
        task.run();
    }

    public void runAll(Task[] tasks) {
        for (Task task : tasks) {
            run(task);
        }
    }
}
```



```java
class CountingRunner extends Runner {
    private int count;

    // ...

    @Override public void run(Task task) {
        count++;
        super.run(task);
    }

    @Override public void runAll(Task[] tasks) {
        count += tasks.length;
        super.runAll(tasks);
    }
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

Bases du langage

Composition vs. héritage : héritage

```java
class Runner {
    // ...

    public void run(Task task) {
        task.run();
    }

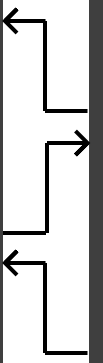
    public void runAll(Task[] tasks) {
        for (Task task : tasks) {
            run(task);
        }
    }
}
```

```java
class CountingRunner extends Runner {
    private int count;

    // ...

    @Override public void run(Task task) {
        count++;
        super.run(task);
    }

    @Override public void runAll(Task[] tasks) {
        count += tasks.length;
        super.runAll(tasks);
    }
}
```



```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

Comptera chaque tache deux fois, à cause de dynamic dispatch.

Dépend de l'implémentation de `Runner` : si `runAll` appelait `task.run()`, pas de problème. Risques de bugs étranges, dus au fort couplage.

Bases du langage

Composition vs. héritage : composition

```go
type Runner struct{}

func (r Runner) Run(t Task) {
    t.Run()
}

func (r Runner) RunAll(ts []Task) {
    for _, t := range ts {
        r.Run(t)
    }
}
```

```go
type CountingRunner struct {
    runner Runner
    count  int
}

func (r *CountingRunner) Run(t Task) {
    r.count++
    r.runner.Run(t)
}

func (r *CountingRunner) RunAll(ts []Task) {
    r.count += len(ts)
    r.runner.RunAll(ts)
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

Bases du langage

Composition vs. héritage : composition

```go
type Runner struct{}

func (r Runner) Run(t Task) {
    t.Run()
}

func (r Runner) RunAll(ts []Task) {
    for _, t := range ts {
        r.Run(t)
    }
}
```

```go
type CountingRunner struct {
    runner Runner
    count  int
}

func (r *CountingRunner) Run(t Task) {
    r.count++
    r.runner.Run(t)
}

func (r *CountingRunner) RunAll(ts []Task) {
    r.count += len(ts)
    r.runner.RunAll(ts)
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

Bases du langage

Composition vs. héritage : composition

```go
type Runner struct{}

func (r Runner) Run(t Task) {
    t.Run()
}

func (r Runner) RunAll(ts []Task) {
    for _, t := range ts {
        r.Run(t)
    }
}
```

```go
type CountingRunner struct {
    runner Runner
    count  int
}

func (r *CountingRunner) Run(t Task) {
    r.count++
    r.runner.Run(t)
}

func (r *CountingRunner) RunAll(ts []Task) {
    r.count += len(ts)
    r.runner.RunAll(ts)
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

runner.runAll(tasks);
```

Bases du langage

Composition vs. héritage : composition

```go
type Runner struct{}

func (r Runner) Run(t Task) {
    t.Run()
}

func (r Runner) RunAll(ts []Task) {
    for _, t := range ts {
        r.Run(t)
    }
}
```

```go
type CountingRunner struct {
    runner Runner
    count  int
}

func (r *CountingRunner) Run(t Task) {
    r.count++
    r.runner.Run(t)
}

func (r *CountingRunner) RunAll(ts []Task) {
    r.count += len(ts)
    r.runner.RunAll(ts)
}
```

```java
CountingRunner runner = new CountingRunner();

Task[] tasks = // ...

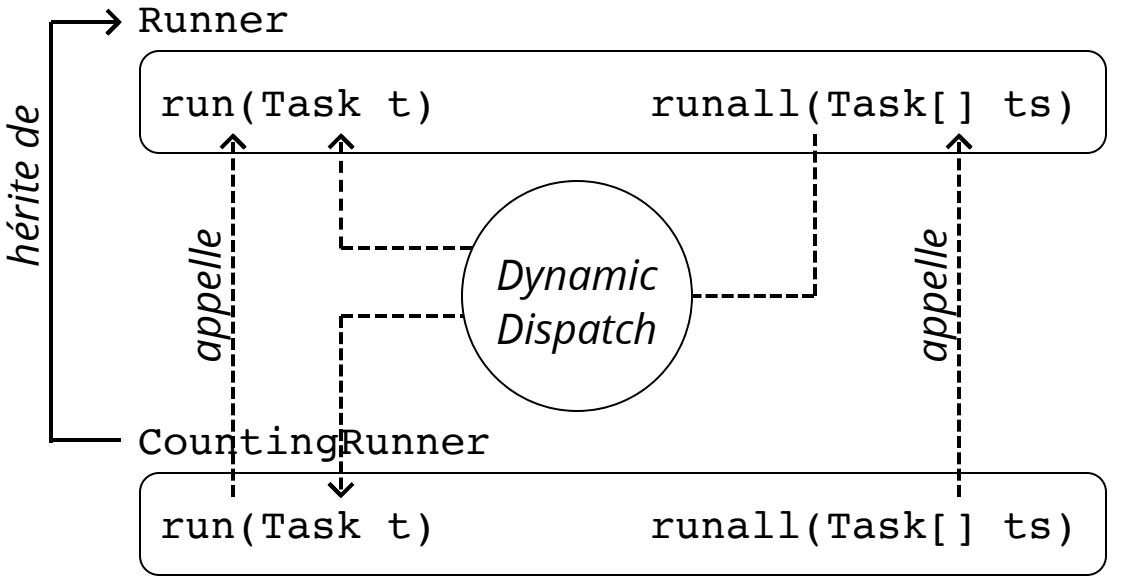
runner.runAll(tasks);
```

Comptera chaque tache une seule fois : plus de dynamic dispatch.

Couplage faible, simplicité accrue, moins de bugs.

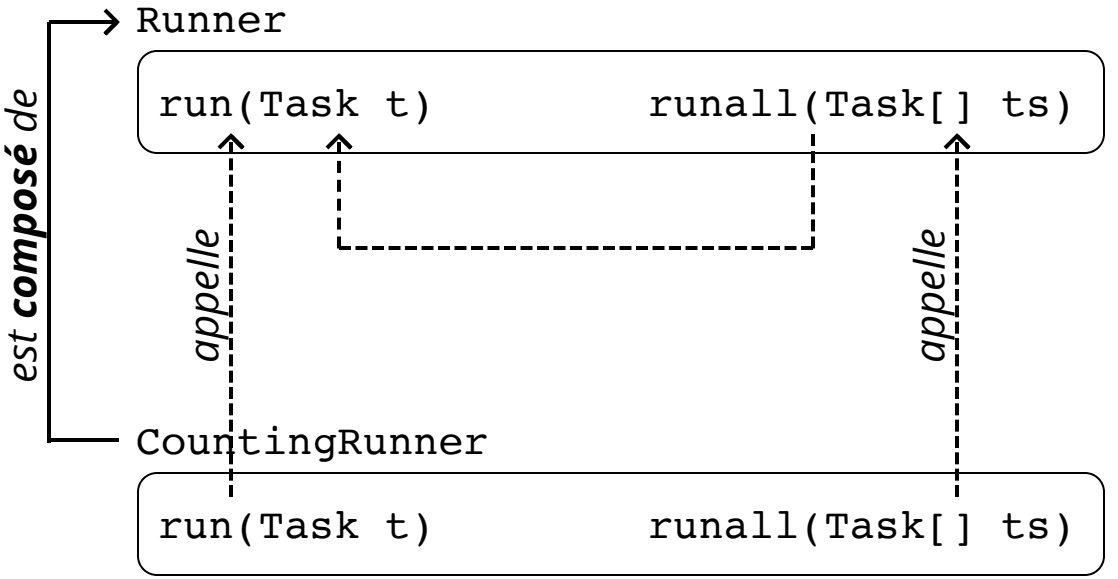
Bases du langage

Composition vs. héritage : héritage



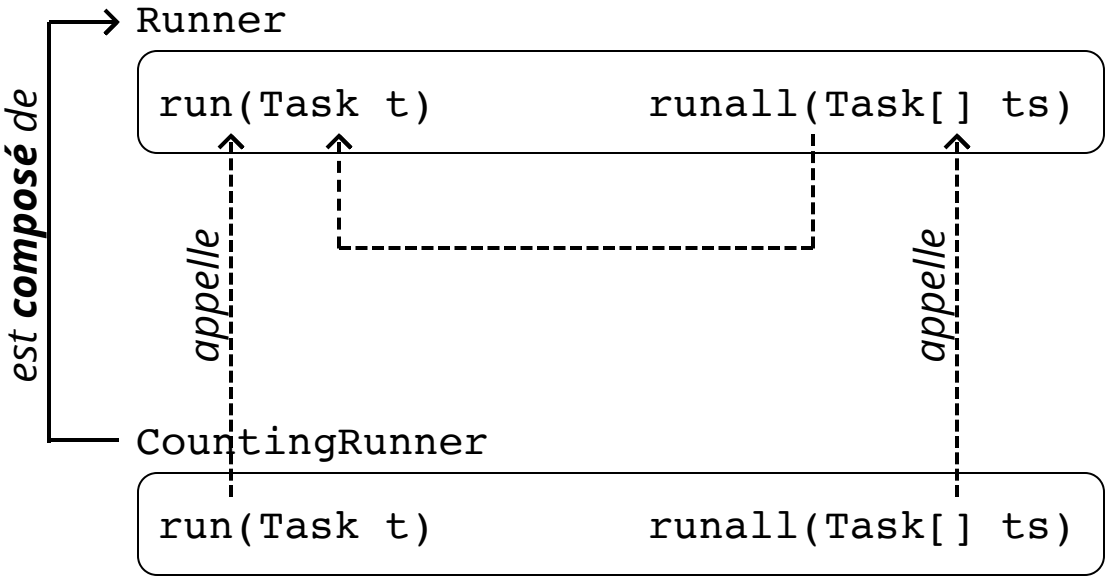
Bases du langage

Composition vs. héritage : composition



Bases du langage

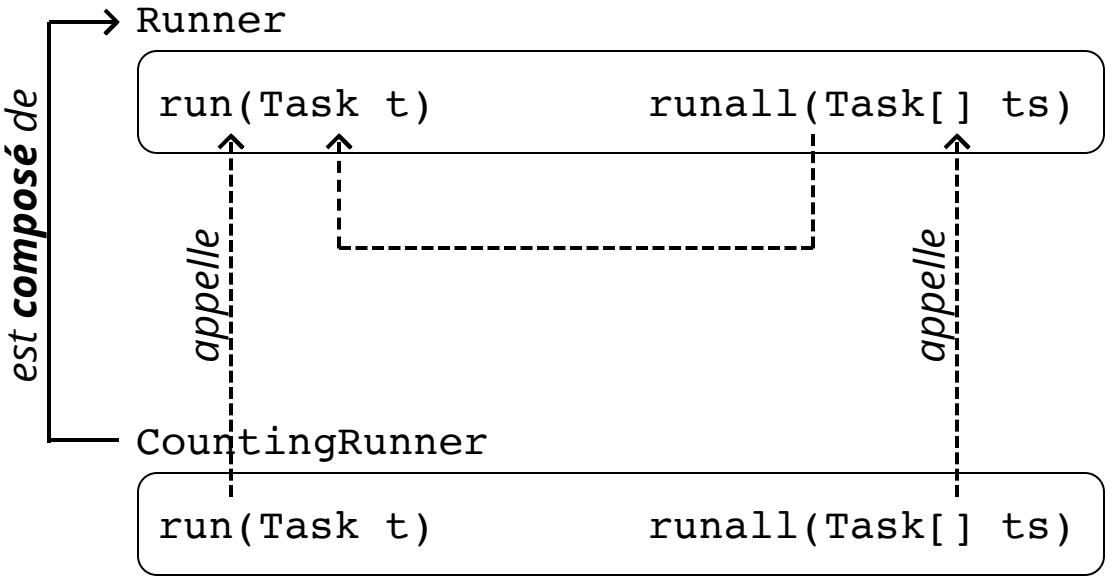
Composition vs. héritage : composition



- Simplification des dépendances : `Runner` est indépendant de `CountingRunner`
- `Runner` peut être créé après `CountingRunner`.
- `Runner` peut être remplacé facilement par un mock pour les tests.

Bases du langage

Composition vs. héritage : composition



- Simplification des dépendances : `Runner` est indépendant de `CountingRunner`
- `Runner` peut être créé après `CountingRunner`.
- `Runner` peut être remplacé facilement par un mock pour les tests.

Mais, on doit réécrire ce qu'on aimerait simplement hériter. *Or do we...*

Bases du langage

Struct embedding

```
type Employee struct {
    Person

    employeeId int
}
```

Bases du langage

Struct embedding

```
type Employee struct {
    Person

    employeeId int
}
```

Si un champ d'une struct n'est qu'un type, on dit qu'il est **incorporé** à cette struct.

Bases du langage

Struct embedding

```
type Employee struct {
    Person

    employeeId int
}
```

Si un champ d'une struct n'est qu'un type, on dit qu'il est **incorporé** à cette struct.

- Un champ est implicitement créé avec comme nom le type incorporé

```
var e Employee
e.Person.Name // Champ de Person
```

Bases du langage

Struct embedding

```
type Employee struct {
    Person

    employeeId int
}
```

Si un champ d'une struct n'est qu'un type, on dit qu'il est **incorporé** à cette struct.

- Un champ est implicitement créé avec comme nom le type incorporé

```
var e Employee
e.Person.Name // Champ de Person
```

- Tous les champs et méthodes du type incorporé sont accessibles directement

```
var e Employee
e.Name      // Champ de Person
e.GetName() // Méthode de Person
```

Bases du langage

Struct embedding

```
type Employee struct {
    Person

    employeeId int
}
```

Si un champ d'une struct n'est qu'un type, on dit qu'il est **incorporé** à cette struct.

- Un champ est implicitement créé avec comme nom le type incorporé

```
var e Employee
e.Person.Name // Champ de Person
```

- Tous les champs et méthodes du type incorporé sont accessibles directement

```
var e Employee
e.Name      // Champ de Person
e.GetName() // Méthode de Person
```

- Similaire à de l'héritage, mais **l'incorporé reste indépendant de l'incorporant**.

Bases du langage

Struct embedding

Exemple - sans struct embedding

```
type Runner struct {
    name string
}

func (r Runner) GetName() string { return r.name }

// ...
```

```
type CountingRunner struct {
    runner Runner
    count int
}

// Obligé de réécrire GetName qui ne fait qu'appeler son parent
func (r CountingRunner) GetName() string { return r.runner.GetName() }

// ...
```

Bases du langage

Struct embedding

Exemple - sans struct embedding

```go
type Runner struct {
    name string
}

func (r Runner) GetName() string { return r.name }

// ...
```

```go
type CountingRunner struct {
    Runner
    count int
}

// Plus nécessaire : automatiquement défini sur CountingRunner
// func (r CountingRunner) GetName() string { return r.runner.GetName() }

// ...
```

Bases du langage

Struct embedding

Autre exemple

```go
type Person struct {
    Name string
}

func (p Person) Introduce() {
    fmt.Println("Hi, I'm ", p.Name)
}


type Employee struct {
    Person
    EmployeeId int
}

func (e Employee) GetEmail() string {
    return e.Name + "@entreprise.ch"
}


func main() {
    e := Employee{Person: Person{"Peter"}, EmployeeId: 123}

    e.Introduce() // Fonctionne, et affiche "Hi, I'm Peter"
}
```

Bases du langage

Struct embedding

Autre exemple

```
 1 type Person struct {
 2     Name string
 3 }
 4
 5 func (p Person) Introduce() {
 6     fmt.Println("Hi, I'm ", p.Name)
 7 }
 8
 9
10 type Employee struct {
11     Person
12     EmployeeId int
13 }
14
15 func (e Employee) GetEmail() string {
16     return e.Name + "@entreprise.ch"
17 }
18
19
20 func main() {
21     e := Employee{Person: Person{"Peter"}, EmployeeId: 123}
22
23     e.Introduce() // Fonctionne, et affiche "Hi, I'm Peter"
24 }
```

Bases du langage

Struct embedding

Autre exemple

```go
type Person struct {
    Name string
}

func (p Person) Introduce() {
    fmt.Println("Hi, I'm ", p.Name)
}


type Employee struct {
    Person
    EmployeeId int
}

func (e Employee) GetEmail() string {
    return e.Name + "@entreprise.ch"
}


func main() {
    e := Employee{Person: Person{"Peter"}, EmployeeId: 123}

    e.Introduce() // Fonctionne, et affiche "Hi, I'm Peter"
}
```

Bases du langage

Struct embedding

Autre exemple

```
 1 type Person struct {
 2     Name string
 3 }
 4
 5 func (p Person) Introduce() {
 6     fmt.Println("Hi, I'm ", p.Name)
 7 }
 8
 9
10 type Employee struct {
11     Person
12     EmployeeId int
13 }
14
15 func (e Employee) GetEmail() string {
16     return e.Name + "@entreprise.ch"
17 }
18
19
20 func main() {
21     e := Employee{Person: Person{"Peter"}, EmployeeId: 123}
22
23     e.Introduce() // Fonctionne, et affiche "Hi, I'm Peter"
24 }
```



Bases du langage

Conversion de types

Bases du langage

Conversion de types

Syntaxe d'une conversion de type (cast) statique

```
<new_type>(<value>)
```

Exemples

```
var x float64 = 3.6
var n int = 2

n = int(x)     // Avec troncation => 2
x = float64(n) // => 2.0
```

```
s := "aéb"      // string, utf8 par défaut, 32 bits car 'é' prend 2 bytes
b := []byte(s)  // Conversion vers slice de bytes : longueur 4
s2 := string(b) // Conversion vers string à nouveau
r := []rune(s)  // Conversion vers slice d'unicode : longueur 3
```

- Pas de conversion implicite en Go.
- Une conversion de type statique n'échouera jamais pendant exécution.

Bases du langage

Conversion de types

Syntaxe d'une conversion de type (cast) dynamique, ou "interface conversion".

```
<value>.(<type>)
```

Version risquée

```
var w Walker = Person{}

var p = w.(Person) // Panic si w pas Person
```

Version safe

```
var w Walker = Person{}

var p, ok = w.(Person)

if ok {
    // Conversion réussie
} else {
    // Conversion échouée
}
```

Bases du langage

Switch sur type

Syntax de conversion dynamique réutilisée pour switch sur type

```
switch w.(type) {
case Person:
    // ...
case Walker:
    // ...
}
```



Bases du langage

Pointeurs

Bases du langage

Pointeurs

Syntaxe du type d'un pointeur

```
*<type> // Pointeur sur une valeur de type <type>
```

Obtenir un pointeur sur une valeur

```
&<variable> // Pointeur sur <variable>
```

Déréférencer un pointeur

```
*<pointeur> // Valeur pointée par <pointeur>
```

Création d'un pointeur sur variable anonyme

```
new(<type>) // Allocation de mémoire de type <type> et création d'un pointeur dessus.
```

Exemples

```
p := new(int)   // Pointeur sur int, type *int
fmt.Println(*p) // Déréférencement du pointeur

func newIntPointer() *int {
    var dummy int
    return &dummy
}
p := newIntPointer() // Pointeur sur int, type *int
// Pas de dangling pointer : Go sait qu'il doit garder dummy en mémoire.
```

Bases du langage

Pointeurs sur struct

Le récepteur d'une méthode est un paramètre comme un autre : peut être un pointeur

```
func (p *Person) IncAge() {
    p.age++
}
```

Le récepteur **peut alors être nul !** Parfois utile.

```
func (p *Person) Name() string {
    if p == nil {
        return "anonymous"
    }
    return p.name
}
```

Le récepteur est implicitement référencé si besoin.

```
jeff := Person{}
jeff.IncAge() // Valide, même si jeff n'est pas un pointeur
```

Accéder aux champs d'une struct pointée se fait normalement (pas de `'->'` comme C)

```
p := new(Person)
p.name = "Johny"
```



Concurrence

Concurrence

"Do not communicate by sharing memory;
instead, share memory by communicating."

- Rob Pike

Concurrence

Goroutines

Définition : fonctionnellement comme un thread, mais plus léger :
démarre avec **peu de mémoire** et alloue plus si besoin,
peut exister par **centaines de milliers**,
builtin à Go.

Syntaxe de lancement d'une goroutine

```
go <statement> // Execute <statement> dans une goroutine
```

Exemple

```
go fetchDocument(documentName)
```

Techniquement : Go réparti les goroutines sur les threads de l'OS.
Une goroutine bloquée ne bloque pas de thread.

Attention : si `main` termine, toutes les goroutines sont tuées.

Concurrence

Channels

Définition : Canal de communication concurrente avec buffer statique.

Syntaxe de création d'un channel

```
make(chan <type>, <cap>) // Crée un channel de type <type>, avec un buffer de taille <cap>
```

Optionnel : si omis, buffer de taille nulle.

Syntaxe d'envoi de données sur un channel

```
<channel> <- <value> // Envoi de <value> de type correspondant sur le channel c
```

Bloquant si le buffer est plein (ou de taille nulle si aucune goroutine en attente de lecture)

Syntaxe de réception de données d'un channel

```
<-<channel> // Réception d'une valeur depuis le channel <channel>
```

Bloquant si le buffer est vide (ou de taille nulle si aucune goroutine en attente d'écriture)

Exemple

```
c := make(chan string, 1)
c <- "Hello, receiver!"
s := <-c
fmt.Println(s) // Hello, receiver!
```

Concurrence

Channels

Définition : Canal de communication concurrente avec buffer statique.

Syntaxe de création d'un channel

```
make(chan <type>, <cap>) // Crée un channel de type <type>, avec un buffer de taille <cap>
```

Optionnel : si omis, buffer de taille nulle.

Syntaxe d'envoi de données sur un channel

```
<channel> <- <value> // Envoi de <value> de type correspondant sur le channel c
```

Bloquant si le buffer est plein (ou de taille nulle si aucune goroutine en attente de lecture)

Syntaxe de réception de données d'un channel

```
<-<channel> // Réception d'une valeur depuis le channel <channel>
```

Bloquant si le buffer est vide (ou de taille nulle si aucune goroutine en attente d'écriture)

Restrictions sur un channel

```
chan<- string
```

Type d'un write-only channel

```
<-chan string
```

Type d'un read-only channel

Concurrence

Channels - fermeture

Syntaxe de fermeture d'un channel

```
close(<channel>)
```

Seul un envoyeur est autorisé à fermer un channel.
Un channel n'est pas comme un fichier :

Pas besoin de le fermer, sauf pour avertir le récepteur qu'il n'y aura pas d'autre message.

Syntaxe de réception de fermeture

```
v, ok := <-c
```

`false` ssi le channel a été fermé par un envoyeur.

Concurrence

Channels - range

Permet de lire un channel jusqu'à ce qu'il soit fermé.

```
for <msg> := range <channel> {
    // do stuff
}
```

≡

```
for {
    <msg>, ok := <-channel>
    if ok {
        // do stuff
    } else {
        break
    }
}
```

Concurrence

Channels - Select

Permet de

- attendre plusieurs channels, et
- optionnellement, les rendre non-bloquants

Syntaxe du `select`

```
select {
case msg := <-chan1:
    // chan1 had a message, do something with it
case msg := <-chan2:
    // chan2 had a message, do something with it
default:
    // neither had a message
}
```

`default` est optionnel : si omis, le select sera bloquant.

Note : Seule une branche est exécutée, même si plusieurs channels ont un message.
On met donc souvent le `select` dans une boucle.

Concurrence

Defer

Permet de repousser l'exécution d'un statement à la fin de la fonction.

```
defer <statement>
```

Exemple

```
func main() {
    conn, err := net.Dial("udp", srvAddr)

    defer conn.Close()

    // ...
}
```

Garantie que la connexion sera fermée en fin de programme.

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version simple, séquentielle

```
1 func makeQueriesSequential(queries []string) (results []Result) {
2     for _, q := range queries {
3         results = append(results, makeQuery(q))
4     }
5     return
6 }
```

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version parallèle

```go
func makeQueriesParallel(queries []string) (results []Result) {
    c := make(chan Result)
    for _, q := range queries {
        go func() { c <- makeQuery(q) }()
    }

    for range queries {
        result := <-c
        results = append(results, result)
    }
    return
}
```

Depuis Go 1.22, `q` est une variable par itération : la capture est correcte. Avant 1.22, les goroutines partageaient la même variable.

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version parallèle

```go
func makeQueriesParallel(queries []string) (results []Result) {
    c := make(chan Result)
    for _, q := range queries {
        go func() { c <- makeQuery(q) }()
    }

    for range queries {
        result := <-c
        results = append(results, result)
    }
    return
}
```

Depuis Go 1.22, `q` est une variable par itération : la capture est correcte. Avant 1.22, les goroutines partageaient la même variable.

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version parallèle

```
 1 func makeQueriesParallel(queries []string) (results []Result) {
 2     c := make(chan Result)
 3     for _, q := range queries {
 4         go func() { c <- makeQuery(q) }()
 5     }
 6
 7     for range queries {
 8         result := <-c
 9         results = append(results, result)
10     }
11     return
12 }
```

Depuis Go 1.22, `q` est une variable par itération : la capture est correcte. Avant 1.22, les goroutines partageaient la même variable.

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version parallèle++

```
 1 func makeQueriesTimeout(queries []string) (results []Result) {
 2     c := make(chan Result, len(queries))
 3     for _, q := range queries {
 4         go func() { c <- makeQuery(q) }()
 5     }
 6
 7     timeout := time.After(80 * time.Millisecond)
 8     for range queries {
 9         select {
10         case result := <-c:
11             results = append(results, result)
12         case <-timeout:
13             fmt.Println("Timed out")
14             return
15         }
16     }
17     return
18 }
```

Note : on donne un buffer non-vide, pour que les goroutines n'attendent pas éternellement en cas de timeout.

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version parallèle++

```
 1 func makeQueriesTimeout(queries []string) (results []Result) {
 2     c := make(chan Result, len(queries))
 3     for _, q := range queries {
 4         go func() { c <- makeQuery(q) }()
 5     }
 6
 7     timeout := time.After(80 * time.Millisecond)
 8     for range queries {
 9         select {
10         case result := <-c:
11             results = append(results, result)
12         case <-timeout:
13             fmt.Println("Timed out")
14             return
15         }
16     }
17     return
18 }
```

Note : on donne un buffer non-vide, pour que les goroutines n'attendent pas éternellement en cas de timeout.

Concurrence

Exemple

Supposons l'existence de `makeQuery(string) Result`
Créons `makeQueries` qui gère plusieurs requêtes.

Version parallèle++

```
 1 func makeQueriesTimeout(queries []string) (results []Result) {
 2     c := make(chan Result, len(queries))
 3     for _, q := range queries {
 4         go func() { c <- makeQuery(q) }()
 5     }
 6
 7     timeout := time.After(80 * time.Millisecond)
 8     for range queries {
 9         select {
10         case result := <-c:
11             results = append(results, result)
12         case <-timeout:
13             fmt.Println("Timed out")
14             return
15         }
16     }
17     return
18 }
```

Note : on donne un buffer non-vide, pour que les goroutines n'attendent pas éternellement en cas de timeout.

Concurrence

Exemple 2

Web page counting its users

```
 1 var nextId = make(chan int)
 2
 3 func handler(w http.ResponseWriter, q *http.Request) {
 4     fmt.Fprintf(w, "<h1>You have id %v<h1>", <-nextId)
 5 }
 6
 7 func main() {
 8     http.HandleFunc("/next", handler)
 9     go func() {
10         for i := 0; ; i++ {
11             nextId <- i
12         }
13     }()
14     http.ListenAndServe("localhost:8080", nil)
15 }
```

Pas besoin de compteur partagé, ni de lock, le channel fait tout pour nous !

Concurrence

Bonnes pratiques

Gestion d'état dynamique

Aucun état ne doit être partagé entre goroutines.

Un état n'existe donc généralement qu'au sein d'une même goroutine.

```
 1 type User struct {
 2     name string
 3     age  int
 4 }
 5
 6 func handleUsers(newUsers <-chan User, leavingUsers <-chan User) {
 7     users := make(map[string]User)
 8     for {
 9         select {
10         case u := <-newUsers:
11             users[u.name] = u
12         case u := <-leavingUsers:
13             delete(users, u.name)
14         }
15     }
16 }
17
18 func main () {
19     go handleUsers(/* ... */)
20 }
```

La gestion de cet état est *sa responsabilité*.

Concurrence

Bonnes pratiques

Obtention d'état dynamique

Pour extraire une information de cet état, on peut fournir un channel de retour.

```
 1 type getUserReq struct {
 2     name   string
 3     result chan<- *User
 4 }
 5
 6 func handleUsers(/* ... */, getUser <-chan getUserReq) {
 7     users := make(map[string]User)
 8     for {
 9         select {
10         /* ... */
11         case req := <-getUser:
12             if u, ok := users[req.name]; ok {
13                 req.result <- &u
14             } else {
15                 req.result <- nil
16             }
17         }
18     }
19 }
```

Concurrence

Bonnes pratiques

Communication

Les messages ne sont délivrés qu'une fois.
Si plusieurs lecteurs sur un même channel, lequel recevra le prochain message est imprévisible.

Donc pour sérialiser : un seul lecteur
pour dispatcher arbitrairement : plusieurs lecteurs

Par contre, plusieurs goroutines peuvent écrire en parallèle sur un même channel.



TCP-IP

Programmation TCP-IP en Go

Client UDP

```
 1 const srvAddr = "127.0.0.1:6000"
 2
 3 func main() {
 4     conn, err := net.Dial("udp", srvAddr)
 5     if err != nil {
 6         log.Fatal(err)
 7     }
 8     defer conn.Close()
 9
10     // Link stdin and stdout to the connection
11     go forward(os.Stdout, conn)
12     forward(conn, os.Stdin)
13 }
14
15 // Links output of src to input of dst
16 func forward(dst io.Writer, src io.Reader) {
17     if _, err := io.Copy(dst, src); err != nil {
18         log.Fatal(err)
19     }
20 }
```

Programmation TCP-IP en Go

Serveur UDP

```go
const srvAddr = "127.0.0.1:6000"

func main() {
    // Start listening on srvAddr
    addr, _ := net.ResolveUDPAddr("udp", srvAddr)
    conn, err := net.ListenUDP("udp", addr)
    if err != nil {
        log.Fatal(err)
    }
    defer conn.Close()

    buf := make([]byte, 1024)
    for {
        // Get data from client
        n, cliAddr, err := conn.ReadFromUDP(buf)
        if err != nil {
            log.Fatal(err)
        }

        // Echo data back
        s := string(buf[0:n]) + " received from " + cliAddr.String() + "\n"
        if _, err := conn.WriteTo([]byte(s), cliAddr); err != nil {
            log.Fatal(err)
        }
    }
}
```

Programmation TCP-IP en Go

Client TCP

```
 1 const srvAddr = "127.0.0.1:8000"
 2
 3 func main() {
 4     conn, err := net.Dial("tcp", srvAddr)
 5     if err != nil {
 6         log.Fatal(err)
 7     }
 8     defer conn.Close()
 9
10     // Link stdin and stdout to the connection
11     go forward(os.Stdout, conn)
12     forward(conn, os.Stdin)
13 }
14
15 // Links output of src to input of dst
16 func forward(dst io.Writer, src io.Reader) {
17     if _, err := io.Copy(dst, src); err != nil {
18         log.Fatal(err)
19     }
20 }
```

Programmation TCP-IP en Go

Client TCP

```
 1 const srvAddr = "127.0.0.1:8000"
 2
 3 func main() {
 4     conn, err := net.Dial("tcp", srvAddr)
 5     if err != nil {
 6         log.Fatal(err)
 7     }
 8     defer conn.Close()
 9
10     // Link stdin and stdout to the connection
11     go forward(os.Stdout, conn)
12     forward(conn, os.Stdin)
13 }
14
15 // Links output of src to input of dst
16 func forward(dst io.Writer, src io.Reader) {
17     if _, err := io.Copy(dst, src); err != nil {
18         log.Fatal(err)
19     }
20 }
```

Programmation TCP-IP en Go

Serveur TCP

```
1 const srvAddr = "127.0.0.1:8000"
2
3 type clientOutChan chan<- string // per-client outgoing messages
4
5 var (
6     joiningClients = make(chan clientOutChan) // New clients to be registered
7     leavingClients = make(chan clientOutChan) // Leaving clients to be deleted
8     broadcast      = make(chan string) // Messages destined to all clients
9 )
```

Programmation TCP-IP en Go

Serveur TCP

```
10
11 func main() {
12     listener, err := net.Listen("tcp", srvAddr)
13     if err != nil {
14         log.Fatal(err)
15     }
16     defer listener.Close()
17
18     // Broadcast to all connection
19     go broadcaster()
20
21     // Wait for new connections
22     for {
23         conn, err := listener.Accept()
24         if err != nil {
25             log.Print(err)
26         } else {
27             go handleConn(conn)
28         }
29     }
30 }
31
```

Programmation TCP-IP en Go

Serveur TCP

```
31
32 func broadcaster() {
33     clientOuts := make(map[clientOutChan]bool) // All connected clients
34
35     for {
36         select {
37         case cli := <-joiningClients:
38             clientOuts[cli] = true
39
40         case cli := <-leavingClients:
41             delete(clientOuts, cli)
42             close(cli)
43
44         case msg := <-broadcast:
45             for cli := range clientOuts {
46                 cli <- msg
47             }
48         }
49     }
50 }
51
```

Programmation TCP-IP en Go

Serveur TCP

```
51
52 func handleConn(conn net.Conn) {
53     defer conn.Close()
54
55     client := make(chan string)
56
57     // Forward messages from client channel to connection
58     go func() {
59         for msg := range client {
60             fmt.Fprintln(conn, msg)
61         }
62     }()
63
64     // Send welcome message to client
65     who := conn.RemoteAddr().String()
66     client <- "You are " + who
67
68     // Broadcasting arrival of new client
69     broadcast <- who + " has arrived"
70
71     joiningClients <- client
72
73     // Broadcasting client messages
74     input := bufio.NewScanner(conn)
75     for input.Scan() {
76         broadcast <- who + " says: " + input.Text()
77     }
78
79     // No more messages, client left.
80     leavingClients <- client
81     broadcast <- who + " has left"
82 }
```

Programmation TCP-IP en Go

Serveur TCP

